

PRG1203

SPACESHOOTERHD

Object-Oriented Programming Project Report

2D Java Space Shooter Application

Presentation video link: <https://www.youtube.com/watch?v=rdkThA3DcJI>

Report information	Details
Course	PRG1203: Object Oriented Programming Fundamentals
Academic session	2026; Semester 2,3
Group number	Group 44
Submission date	11 August 2026

TEAM MEMBERS

No.	Name	Student ID
1	Muhammad Husein Ilhamsyah	2511028
2	Abdullah Asghar	25088261
3	Clara Annabelle Widjaya	24153751
4	Wayne Bertrand Lesperance	25018177

1. System Description

SpaceshooterHD is a 2D arcade space shooter game built in Java using Swing for its tools of rendering and windowing. The player/user controls a spaceship using WASD for movement, Space bar for firing bullets, while maneuvering to evade enemy projectiles and asteroids and accumulating points. A full menu system (main menu, instructions, controls, and a game-over screen) before and after gameplay. The game runs on a fixed-timestep loop on a dedicated thread, decoupled from Swing's EDT (Event Dispatch Thread), so the gameplay logic and rendering remain smooth and frame rate independent.

1.1 Core Gameplay Loop

The game is driven by GamePanel which implements Runnable and own specific game threads. Each frame follows the same three-step cycle:

- `GameClock.waitForNextFrame()` blocks until the next frame is due (targeting 60 FPS) and returns `deltaTime` which is the elapsed time in seconds since the previous frame.
- `GameController.update(deltaTime)` advances all game logic: menu/state handling, player movement, shooting, spawning, AI's decisions and collision detection.
- `Repaint()` triggers Swing's `paintComponent()` which delegates all drawing to `GameController.render(Graphics2D)`

This separation of making the logic thread distinct from the rendering thread is why several collections in `GameController` (bullets, asteroids, enemies, explosions, powerups, stars) are wrapped in synchronized blocks: `update()` mutates them from the game thread while `render()` reads them from the EDT.

1.2 Menu & Game State System

- `GameController` works now as a simple state machine with an internal enum `GameState`: `MENU`, `INSTRUCTIONS`, `CONTROLS`, `GAME_OVER`, and `PLAYING`. Both `update()` and `render()` check the current state — when not `PLAYING`, the game renders an animated menu scene (scrolling star background, asteroid decorations flying around, shooting stars flying across, and the animated player ship staying still in place).
- `Main.java` has a text menu running concurrently on the console side (with `Scanner`), displaying ASCII logo and a set of numbered choices; choosing a choice triggers calls to `GameFrame` and `GamePanel` and then to `GameController`.

- Menu in the window can also be controlled via number keys (1-4, 0 for returning back) — InputManager keeps a boolean field for pendingMenuChoice, set in keyPressed() and accessible via consumeMenuChoice(), to ensure each menu choice can only be processed once per each press.
- GAME_OVER state is transitioned into after fading in with a fixed time period (gameOverFade, animated with deltaTime), and beginGameplay() initializes player, score and every entities list back to the default state, to make sure that a new game doesn't start with any old bullets or asteroids left from the previous one.
- All drawing of the menu/game-over screens (title, animated ship preview, shooting stars, instructions/controls pages, game-over overlay) is outsourced to a separate class MenuRenderer; GameController makes the decision of what state to display while MenuRenderer decides how to display it. It is a conscious refactoring aimed at keeping render() method more focused on gameplay rendering.

1.3 Implemented Features

Player Movement & Combat

- Inertia-based movement (WASD): acceleration, a maximum speed cap, and friction-based deceleration rather than instant fixed-speed movement; the ship now stops cleanly (zeroing velocity) on hitting a screen edge instead of just clamping position.
- Shooting (Space bar) with a cooldown timer to prevent full-auto fire.
- A temporary "Spread Shot" power-up that fires three angled bullets per shot instead of one, for a limited duration, worth +20 score on pickup.
- A lives system (3 starting lives) with a brief invulnerability window after taking damage, visually communicated by the ship blinking on screen; taking damage now also subtracts 15 points from score.
- The player ship is drawn via an animated 25-frame spritesheet (SpriteSheet + ShipAnimator) that smoothly cycles from an idle loop through four thrust intensity zones based on current speed, with a static-image fallback if the spritesheet fails to load.
- The player's collision hitbox is now a triangular Polygon (nose-and-wings shape) rather than a rectangle, checked via a Shape/Area-based intersection test.

Asteroids

- Spawn periodically from the right edge of the screen at randomised vertical position and speed, drifting left; some travel a straight horizontal line while others also drift vertically and bounce off the top/bottom edges.
- Colliding with the player damages the player (and now applies a score penalty) and destroys the asteroid, with a burst-particle explosion effect and a chance to drop a power-up.
- Asteroids can collide with each other, destroying both and splitting each into two smaller fragments (down to a minimum size), each with a brief post-split collision grace period so fragments don't immediately re-collide.

Enemy Ships

- Two enemy types — GruntEnemyShip and EliteEnemyShip — spawn from the right edge, fly straight left, and fire bullets at the player based on independent AI rules (detailed in Section 2.4).
- Player bullets destroy enemy ships for score; enemy bullets damage the player on contact.
- Enemy ships render an actual sprite image when available, and fall back to a drawn coloured triangle if the image asset fails to load.

Background & Atmosphere

- A persistent scrolling starfield (Star entities, drifting right-to-left, continuously respawned to maintain a constant on-screen count) renders behind all gameplay and menu states alike, over a vertical gradient "deep space" background.
- The main menu features a glowing ASCII-art title, pulsing text, drifting decorative asteroids, animated shooting-star streaks, and the player's own animated ship hovering with engine-trail effects — purely cosmetic, built directly with Graphics2D primitives rather than image assets.

Scoring & HUD

- ScoreManager tracks score, incremented on destroying an asteroid (+10), destroying an enemy (+25), or collecting a power-up (+20), and decremented on taking damage (−15, floored at zero).
- The HUD (drawn directly by GameController.render()) displays current score, remaining lives, active spread-shot duration, and a faded-in "GAME OVER" screen with restart/menu options once lives reach zero.

2. OOP Concepts Used

2.1 Encapsulation

Each class maintains its own internal state privately and provides public behavior via an interface:

- PlayerSpaceship class keeps velocityX, velocityY, lives, bankAngle, and invulnerabilityRemaining variables private. Access from any external classes is impossible, and such methods as takeHit() enforce invulnerability internally and then decrement lives; similarly, only reset() allows to initialize a new instance of a PlayerSpaceship.
- Spaceship class keeps x, y, width, height and image variables protected (not public), and only subclasses can change positions directly by manipulating them, while other properties can be accessed only through getX(), getY(), getBounds(), etc.
- InputManager class completely hides its internal state: APressed, WPressed, pendingMenuChoice and similar are kept private and can be obtained only through boolean functions isAPressed(); one-shot consumeMenuChoice() gets the choice and resets it on return, so the choice cannot be re-read.
- AssetManager and SpriteSheet provide a simplified interface for potentially complex initialization process: loadImage(String path) uses internal caching HashMap<String, BufferedImage>, and SpriteSheet class pre-calculates all its grid slicing in constructor and exposes getFrame(int index).

It should be mentioned that AssetManager currently provides public constructor and static getInstance() method. Encapsulation of the singleton pattern requires making a constructor private to guarantee there will be only one instance of it.

2.2 Inheritance

Inheritance manifests itself in the simplest form in the ship hierarchy:

Spaceship (abstract)-> base properties: x, y, width, height, image; base methods: update() (abstract), render(), getBounds(), getX/Y/Width/Height, getCenterX/Y

- PlayerSpaceship extends Spaceship – includes control input handling, animated spritesheet, lives and blinking when invulnerable. This class has two constructors (one accepting a static

BufferedImage, the other a SpriteSheet), so the same class can use both animations and the fallback static rendering depending on whether the constructor with a spritesheet was called – constructor overloading makes it possible not to care about which constructor was called from outside.

- EnemySpaceship extends Spaceship (abstract too) – implements the common straight-line movement and shooting cooldown time for all enemies.
- GruntEnemyShip extends EnemySpaceship and EliteEnemyShip extends EnemySpaceship – inherit all the common enemy logic and only have their own shooting behavior and sprite animation.

Thus it becomes unnecessary to write position / rendering / bounds logic thrice, and it is guaranteed that any new enemy will be able to work with only one additional method implemented.

2.3 Aggregation & Composition

The project uses both relationship types depending on how tightly an object's lifecycle is tied to its owner:

Composition (whole owns and controls the part's lifetime)

- GameController composes its entity lists - List<Bullet>, List<Asteroid>, List<Explosion>, List<PowerUp>, List<EnemySpaceship>, and recently added List<Star> - for background stars. All these entities are fully managed by GameController object and do not exist on their own outside of it.
- Explosion composes its private static array of Particle objects. Particle is a private inner class which has no independent existence outside the Explosion class.
- PlayerSpaceship composes a ShipAnimator object that is created in the constructor of the spaceship and destroyed with it.
- GameController composes a MenuRenderer object which is created once in the constructor of the GameController and provided with needed images of the player and the asteroids to be used in menu drawing. MenuRenderer does not have any lifetime outside of the controller that created it - nice example of composition used specifically to separate responsibilities of drawing menu/game-over screen from GameController itself.
- GameFrame composes GamePanel that in turn composes GameController, InputManager and GameClock objects created once in GamePanel and GameFrame constructors respectively and not referenced anywhere else.

Aggregation (whole uses the part, but does not own its lifecycle)

- InputManager is created once by GamePanel but is shared into GameController and, in turn, into PlayerSpaceship via constructor injection. GameController and PlayerSpaceship use the same InputManager instance without owning or destroying it so it continues to exist and be updated by the keyboard listener independently of any one class holding a reference to it.
- AssetManager is a singleton aggregated by GameController (and used to load images referenced by Asteroid, PlayerSpaceship, and the enemy ship classes) rather than being constructed and owned by any single class.
- The SpriteSheet built in GameController is passed into PlayerSpaceship's constructor and referenced by it, but GameController retains its own reference (playerSpriteSheet) for potential reuse elsewhere as GameController does not hand off exclusive ownership.

2.4 Polymorphism

Polymorphism is used throughout the ship hierarchy via method overriding, resolved at runtime based on the actual object type rather than the reference type:

- The update(double deltaTime) method is made abstract in Spaceship, while being implemented completely differently by PlayerSpaceship (reading input, applying inertia, moving ship animation) and by EnemySpaceship (moving in straight lines and deciding to shoot).
- The render(Graphics2D) method has a default implementation in Spaceship, which draws the sprite, while being overridden by PlayerSpaceship (checks blink timer, finds animated frame number, applies rotation transformation before drawing) and by GruntEnemyShip/EliteEnemyShip (draws flipped sprite, or alternatively draws a colored polygon when there is no image file loaded).
- The decideToShoot(double enemyCenterY, double playerCenterY) method is abstract in EnemySpaceship class. The GruntEnemyShip decides to shoot "if aligned with player lane or else about 25% probability randomly," while EliteEnemyShip will decide to shoot "if aligned with the player's lane and not otherwise."
- CollisionManager.isColliding(Shape a, Shape b) actually uses polymorphism on the java.awt library level. Rectangle and Polygon classes implement the Shape interface, which means that using the same collision detection method works for the player's triangular hitbox as well as all entities' rectangular bounding boxes, without any overloading or explicit type checking for each shape kind.

The actual effect of this polymorphism is seen in GameController: The loop “for (EnemySpaceship enemy : enemies) { enemy.render(g); }” goes through the list containing different types of GruntEnemyShip and EliteEnemyShip, calling the same method for each of them without using any type checking or conditionals (if statements). Each object will call its own overridden method automatically – this is what dynamic method dispatch is.

3. UML Diagram

The class and class-relationship diagrams below were designed in draw.io and are embedded as figures. Figure 1 shows the application, core game-loop, and system-service packages together with their composition, aggregation, and dependency relationships. Figure 2 shows the entity package in detail — the full Spaceship inheritance hierarchy, gameplay-object ownership by GameController, and multiplicities for every collection GameController maintains.

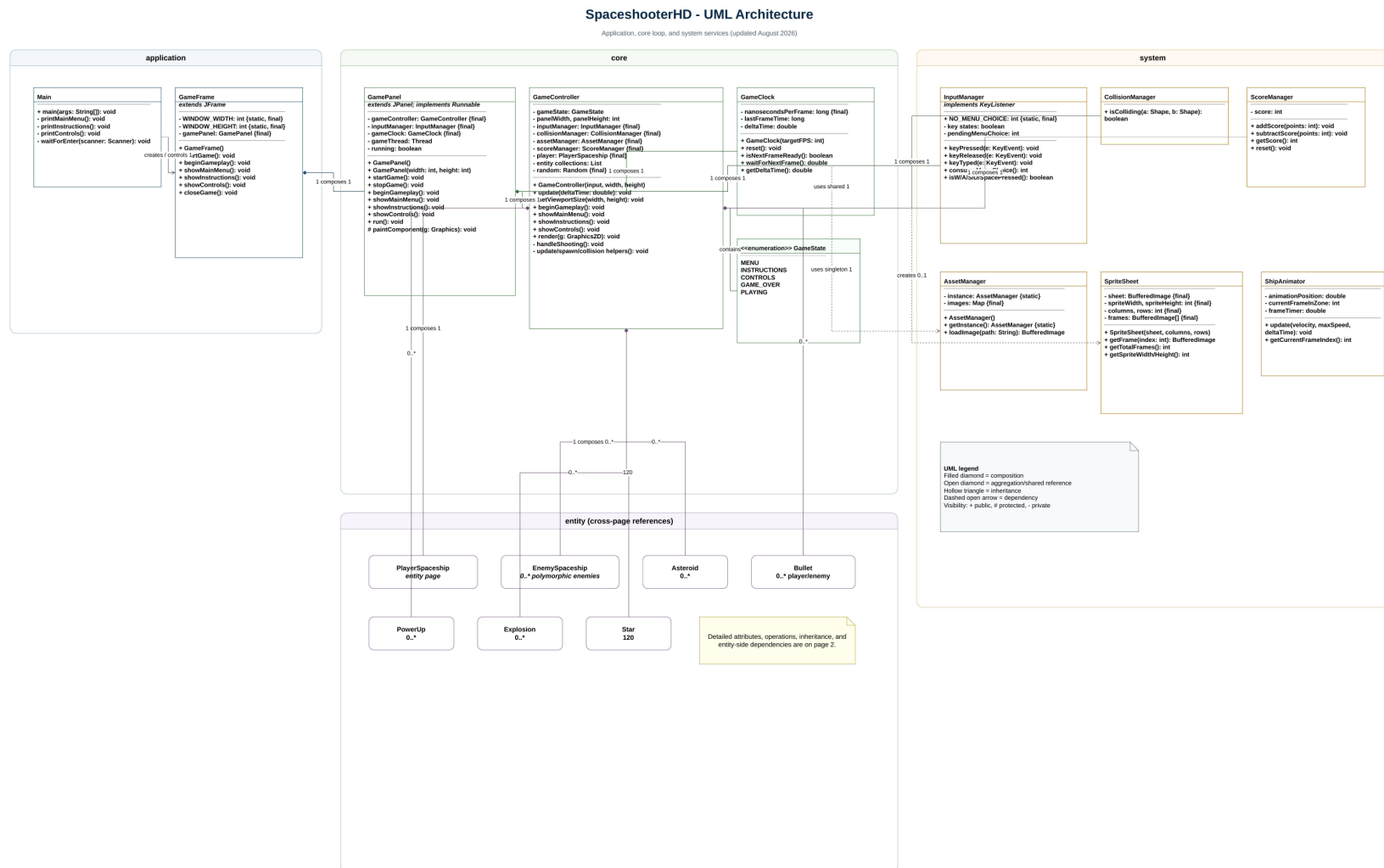


Figure 1 — Application, core loop, and system services

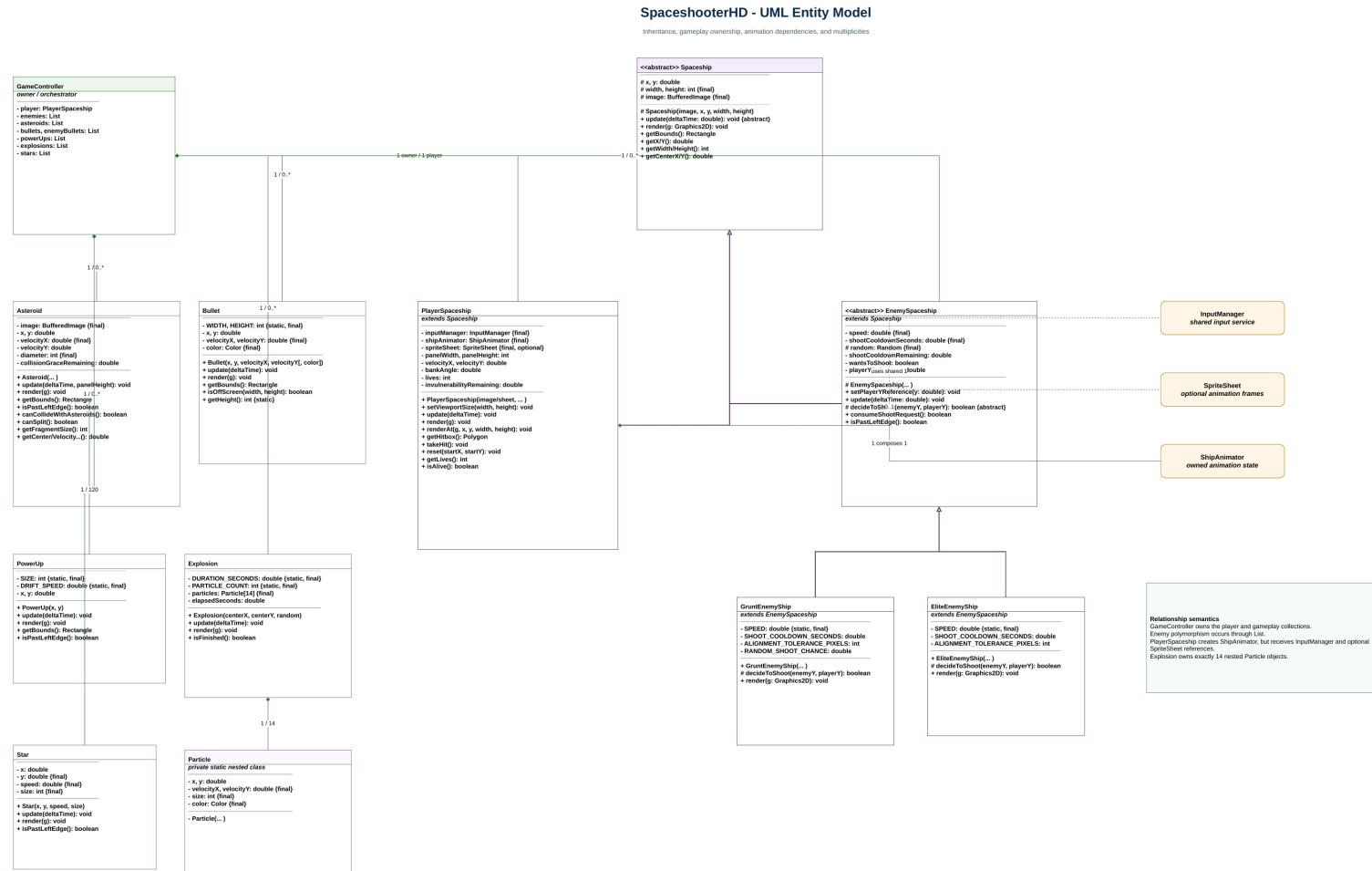


Figure 2 — Entity model: inheritance, ownership, and multiplicities

3.1 Package Overview

- `com.r3m.spaceshooter.application` — `Main`, `GameFrame` (window bootstrap and console-driven menu entry point)
- `com.r3m.spaceshooter.core` — `GamePanel`, `GameClock`, `GameController`, `MenuRenderer` (game loop, orchestration, and non-gameplay screen rendering)
- `com.r3m.spaceshooter.system` — `InputManager`, `CollisionManager`, `AssetManager`, `ScoreManager`, `SpriteSheet`, `ShipAnimator` (cross-cutting services)
- `com.r3m.spaceshooter.entity` — Spaceship hierarchy, `Bullet`, `Asteroid`, `Explosion`, `PowerUp`, `Star` (game objects)

4. Conclusion

In Conclusion, SpaceshooterHD demonstrates and implement Object Oriented Programming concepts and software development rules in general.

This concept includes inheritance, encapsulation, a real-time game loop, delta-time movement, vector-based movement, game-state management, keyboard input, collision detection, resource caching and sprite animation. With this project we learned several new concepts and its implementations, e.g. multiple new libraries, 2D game development and more.

As much of effort we put into this project, several improvements are still available such as, sound effects, increasing difficulty, more enemy types, a boss perhaps and many more.

Lastly as a footnote, the Javadoc documentation for the project is included in the “doc” folder within the submitted zip file. Considerable effort was dedicated to documenting the project. Therefore, if any part of the implementation requires further clarification, please do refer to the Javadoc documentation for additional details.